EXPLORATORY DATA ANALYSIS

Chapter 18

High-Dimensional EDA

Understanding many variables without losing analytical discipline

Course tools: Python, NumPy, Pandas, Matplotlib's object-oriented API, and Seaborn.

This chapter investigates datasets with many variables. It remains focused on exploratory understanding—not feature selection, dimensionality-reduction algorithms, or model training.

Learning objectives

• Explain what makes a dataset high-dimensional and why many-variable analysis is difficult.

• Audit wide datasets for types, missingness, constants, near-constants, identifiers, and duplicate columns.

• Organize variables into meaningful domain groups and measurement types.

• Summarize pairwise relationships without treating correlation as a complete description.

• Investigate missingness patterns across both columns and rows.

• Use heatmaps, clustered displays, pair plots, and small multiples responsibly.

• Recognize scaling, multiple-comparison, and high-dimensional distance limitations.

• Build a reusable audit and a concise professional EDA report.

Core principle: high-dimensional EDA is not about plotting every column against every other column. It is about organizing variables, finding structure, checking data quality, and selecting views that answer specific questions.

1. What does high-dimensional mean?

A dataset is often called high-dimensional when it contains many variables relative to the number of observations, or simply enough variables that ordinary one-variable-at-a-time inspection becomes impractical. There is no universal cutoff. A table with 80 columns and 500 rows may need a different workflow from a table with 5 columns and 500,000 rows.

Represent a data matrix as X ∈ R^(n×p), where n is the number of observational units and p is the number of variables. High-dimensional EDA studies the structure of columns, their dependencies, and the quality of rows across this matrix.

Notation	Meaning	Example
n	Number of observations	2,000 customers
p	Number of variables	150 measurements
X■■	Value of variable j for observation i	Customer 42's monthly spend
p/n	Rough ratio, not a universal rule	150/2,000 = 0.075

Do not confuse high-dimensional with high-volume. Millions of rows and four columns pose different challenges from a few hundred rows and thousands of columns.

2. Why many-variable analysis is difficult

• **Combinatorial growth:** p variables create p(p–1)/2 unique pairs. With 20 variables there are 190 pairs; with 100 there are 4,950.

• **Visual overload:** pair plots and heatmaps can become unreadable.

• **Redundancy:** columns may encode the same quantity or be derived from one another.

• **Mixed types:** numeric, categorical, ordinal, date/time, Boolean, and identifier columns need different summaries.

• **Missingness complexity:** blanks may occur in coordinated blocks or depend on collection processes.

• **Small-sample instability:** many columns and few rows produce noisy correlations and subgroup summaries.

• **Multiple comparisons:** scanning thousands of relationships increases the chance of noticing patterns by chance.

The number of unique pairs grows quadratically. Doubling p produces roughly four times as many pairs. Organize variables before computing or plotting everything.

3. Review of earlier chapters (Chapters 1–14)

This review excludes Chapter 17, which is treated separately below.

• **Ch. 1 — EDA:** exploration is question-driven and iterative; an observed pattern is not automatically an explanation.

• **Ch. 2 — Dataset understanding:** establish observational unit, grain, variable meaning, measurement scale, and data-generating process.

• **Ch. 3 — Data quality:** audit completeness, validity, consistency, uniqueness, accuracy, and timeliness.

• **Ch. 4 — Numerical variables:** use robust and non-robust summaries, quantiles, spread, and shape appropriately.

• **Ch. 5 — Categorical variables:** inspect frequencies, rare levels, cardinality, ordered categories, and denominators.

• **Ch. 6 — Date/time:** understand parsing, time zones, temporal coverage, intervals, and calendar structure.

• **Ch. 7 — Descriptive statistics:** choose summaries by distribution and purpose; distinguish sample and population conventions.

• **Ch. 8 — Distributions:** inspect tails, gaps, multimodality, empirical distributions, histograms, and KDE limitations.

• **Ch. 9 — Shape:** skewness, kurtosis, and quantiles describe shape but do not establish validity.

• **Ch. 10 — Distribution visualization:** use histograms, ECDFs, boxplots, violin plots, and related displays as complementary views.

• **Ch. 11 — Relationships:** inspect form, direction, strength, nonlinearity, overplotting, and confounding.

• **Ch. 12 — Multivariate visualization:** use facets, heatmaps, encodings, and scale choices carefully.

• **Ch. 13 — Association:** choose measures for variable types; correlation is not causation.

• **Ch. 14 — Covariance and dependence:** distinguish zero correlation from independence and inspect conditional structure.

4. Review of Chapter 17 — Investigating Unusual Observations

Chapter 17 established that a detector produces candidates, not verdicts. Investigate source records, units, keys, joins, timestamps, domain constraints, and relevant peer groups. Quantify influence on conclusions, and document corrections, exclusions, or separate treatment.

> **Expected reasoning for the Chapter 17 assignment/project:** preserve the raw observation, define the peer group, collect source evidence, compare the candidate with peers, assess sensitivity, and record a justified decision. If no assignment was submitted, this is a reasoning guide, not a grading result.

In a wide dataset, a row may look ordinary in each column but unusual as a combination. Conversely, a row may be flagged because many fields are missing or measured on incompatible scales.

5. Start with a variable inventory

Before examining correlations, inventory the columns: data types, missingness, unique counts, constants, near-constants, likely identifiers, and possible duplicate columns. This is a screening tool, not an automated verdict.

```
import numpy as np
import pandas as pd

def variable_inventory(df):
    rows = []
    n = len(df)
    for col in df.columns:
        s = df[col]
        non_missing = s.dropna()
        rows.append({
            "column": col,
            "dtype": str(s.dtype),
            "missing_n": int(s.isna().sum()),
            "missing_pct": 100 * s.isna().mean() if n else np.nan,
            "unique_n": int(s.nunique(dropna=True)),
            "constant": s.nunique(dropna=True) <= 1,
            "sample_values": non_missing.head(3).tolist()
        })
    return pd.DataFrame(rows).sort_values(
        ["missing_pct", "unique_n"],
        ascending=[False, True]
    )

inventory = variable_inventory(df)
inventory.head(20)
```

A high unique count does not prove a column is an identifier; a low unique count does not prove it is unimportant. Interpret values using metadata and domain knowledge.

6. Organize variables into meaningful groups

A wide dataset becomes easier to reason about when columns are grouped by meaning, not merely by storage type.

Group	Examples	Useful first-pass EDA
Identifiers	customer_id, order_id, device_id	Uniqueness, duplicates, key integrity
Profile	age_band, region, segment	Frequency, missingness, subgroup coverage
Behavior	visits, spend, clicks	Distributions, rates, relationships
Operations	temperature, pressure, duration	Units, ranges, temporal patterns
Outcomes	returned, churned, defect_flag	Class balance, denominators, subgroup rates
Metadata	created_at, source_system	Coverage, parsing, collection effects

Build a data dictionary with name, definition, unit, type, valid range or categories, source, and analytical role. A column name alone rarely establishes meaning.

7. Data quality patterns across columns

7.1 Missingness by variable

Rank columns by missing count and percentage. Always retain denominators. If columns apply to different populations, percentages may not be comparable.

```
missing_summary = (
    df.isna().agg(["sum", "mean"]).T
      .rename(columns={"sum": "missing_n", "mean": "missing_rate"})
      .sort_values("missing_rate", ascending=False)
)
missing_summary["missing_pct"] = 100 * missing_summary["missing_rate"]
```

7.2 Missingness by row

Some rows may be complete while others have many blank fields. This can reveal incomplete forms, partial migrations, or source systems that collect fewer variables.

```
row_missing = df.isna().sum(axis=1)
row_missing_summary = row_missing.value_counts().sort_index()
df_audit = df.assign(
    missing_fields_n=row_missing,
    missing_fields_pct=100 * df.isna().mean(axis=1)
)
```

7.3 Missingness co-occurrence

Columns missing together may share a collection process. This matrix is descriptive, not proof of a causal mechanism.

```
missing_indicator = df.isna().astype("int8")
missing_cooccurrence = missing_indicator.T @ missing_indicator
```

For large p, inspect a selected subset rather than rendering an unreadable full p-by-p matrix.

8. Constants, near-constants, and redundancy

A constant column has one observed value. A near-constant column has one dominant value and few alternatives. It may reflect a fixed setting, inactive measurement, import issue, or legitimately rare event.

```
def low_variation_report(df, threshold=0.99):
    rows = []
    for col in df.columns:
        s = df[col].dropna()
        if s.empty:
            continue
        proportions = s.value_counts(normalize=True)
        rows.append({
            "column": col,
            "unique_n": s.nunique(),
            "dominant_value": proportions.index[0],
            "dominant_share": proportions.iloc[0],
            "near_constant": proportions.iloc[0] >= threshold
        })
    return pd.DataFrame(rows).sort_values(
        "dominant_share", ascending=False
```

```
    )
```

Redundancy can arise from duplicate columns, unit conversions, totals and component columns, or deterministic formulas. Document such relationships; do not silently drop columns.

8.1 Exact duplicate columns

```
duplicate_pairs = []
cols = list(df.columns)
for i, left in enumerate(cols):
    for right in cols[i + 1:]:
        if df[left].equals(df[right]):
            duplicate_pairs.append((left, right))
duplicate_pairs
```

This pairwise scan requires $O(p^2)$ comparisons and may be expensive. Use it on a screened subset or compare hashes after careful normalization. Equality in this sample does not prove that two fields have identical meanings.

9. Pairwise relationships: useful but incomplete

For p numeric variables, a correlation matrix has p^2 cells but only $p(p–1)/2$ distinct off-diagonal pairs. It is a map of pairwise association, not a complete description of joint dependence.

```
numeric = df.select_dtypes(include="number")
pearson_corr = numeric.corr(method="pearson", min_periods=30)
spearman_corr = numeric.corr(method="spearman", min_periods=30)
```

Pearson emphasizes linear association and can be sensitive to outliers. Spearman measures monotonic association through ranks. Neither is universal for nonlinear dependence, and neither proves causation.

9.1 Pairwise sample size matters

With missing values, each pair may use a different set of rows. Report pairwise counts so coefficients are not compared without sample-size context.

```
valid = numeric.notna().astype("int64")
pairwise_n = valid.T @ valid
```

9.2 Correlation heatmaps

```
import matplotlib.pyplot as plt
import seaborn as sns

fig, ax = plt.subplots(figsize=(11, 9))
sns.heatmap(
    pearson_corr, cmap="vlag", center=0,
    vmin=-1, vmax=1, square=True, ax=ax
)
ax.set_title("Pearson correlation — selected numeric variables")
fig.tight_layout()
```

For large p, labels overlap and patterns become difficult to interpret. Select variables based on a question or domain group. Do not interpret a strong color as a causal relationship.

10. Variable clustering as an exploratory idea

Variables can be grouped by similarity in observed patterns. A clustered correlation heatmap reorders variables so related columns appear near one another.

```
selected_corr = pearson_corr.loc[selected_columns, selected_columns]
cluster_grid = sns.clustermap(
    selected_corr, cmap="vlag", center=0,
    vmin=-1, vmax=1, figsize=(10, 10)
)
```

Clustering depends on distance, linkage, variable selection, missing-data handling, and scaling. The groups are exploratory summaries—not objective natural categories, validated feature groups, or permission to remove variables.

11. Mixed-type relationships

A high-dimensional dataset often mixes numeric, categorical, ordinal, and temporal columns. A single Pearson matrix cannot describe all relationships.

Variable pair	Possible EDA tools	Caution
Numeric–numeric	Scatterplot, Pearson/Spearman, conditional summaries	Nonlinearity, outliers, confounding

Variable pair	Possible EDA tools	Caution
Categorical–categorical	Crosstab, proportions, chi-square, Cramér's V	Sparse cells, denominator choice
Categorical–numeric	Grouped distributions, box/violin/strip plots	Unequal group sizes
Date/time–numeric	Time plots, calendar summaries, lagged descriptions	Trend, seasonality, time definition
Ordinal–numeric	Rank association, ordered summaries	Ordinal spacing may not be equal

Choose association measures according to measurement scale and the question. Revisit Chapters 13–15 for association, dependence, and grouped analysis.

12. Pair plots and small multiples

Pair plots can reveal nonlinear patterns, clusters, and unusual combinations among a small selected set. Do not apply them indiscriminately to hundreds of columns.

```
selected = [
    "monthly_spend", "visit_count",
    "average_order_value", "days_since_last_visit"
]
pair_grid = sns.pairplot(
    df[selected].dropna(), corner=True, diag_kind="hist",
    plot_kws={"alpha": 0.45, "s": 18}
)
```

Choose variables that represent a coherent question. Small multiples are often more readable: one panel per variable group, subgroup, or period, using consistent scales when comparison requires them.

13. The curse of dimensionality — an EDA perspective

In many dimensions, observations become sparse relative to the space they could occupy. A two-dimensional projection may hide structure present in other variables. Distance-based notions can also become less discriminating as dimensionality increases, especially with noisy variables or incompatible scales.

• Do not assume a two-variable projection captures the full structure.

• Account for units when comparing distances, but remember scaling changes geometry.

• Inspect domain-defined groups of related variables.

• Treat high-dimensional outlier scores as method-dependent signals requiring investigation.

• Prefer interpretable summaries and targeted visual slices over a single impressive projection.

This chapter does not teach PCA, UMAP, t-SNE, feature selection, or predictive modeling. Those are separate topics. Here we describe original variables and observed relationships.

14. Multiple comparisons and exploratory over-reading

When thousands of pairs are inspected, some may look strongly related by chance. Distinguish a pattern noticed during exploration from a hypothesis tested on independent data. Record which relationships were explored and seek domain explanations or later validation.

Do not turn a correlation heatmap into causal claims. High association may arise from shared trends, a third variable, selection effects, or deterministic construction.

15. Reusable high-dimensional audit

```
def high_dimensional_audit(df, numeric_min_periods=30):
    n, p = df.shape
    inventory = variable_inventory(df)
    numeric = df.select_dtypes(include="number")

    missing_by_column = (
        df.isna().mean().rename("missing_rate")
          .sort_values(ascending=False)
    )
    missing_by_row = df.isna().mean(axis=1)
    corr = numeric.corr(
```

```
        method="spearman", min_periods=numeric_min_periods
    )
    valid = numeric.notna().astype("int64")
    pairwise_n = valid.T @ valid

    return {
        "shape": {"rows": n, "columns": p},
        "inventory": inventory,
        "missing_by_column": missing_by_column,
        "missing_by_row": missing_by_row,
        "numeric_correlation": corr,
        "pairwise_sample_size": pairwise_n
    }

# audit = high_dimensional_audit(df)
# audit["shape"]
# audit["inventory"].head()
```

This is a starting framework, not an automated quality verdict. Add domain rules, categorical association summaries, temporal coverage, and selected plots according to the dataset's purpose.

16. Common mistakes

• Plotting every pair and assuming more plots mean more thorough analysis.

• Treating correlation as a complete description of dependence.

• Using Pearson correlation for every variable type.

• Ignoring pairwise sample sizes when missingness differs.

• Treating correlated columns as proof that they measure the same construct.

• Removing constant or redundant columns without checking meaning and provenance.

• Comparing raw distances across incompatible units.

• Reading a heatmap as a causal diagram.

• Over-interpreting patterns found after thousands of exploratory comparisons.

• Treating a two-dimensional projection as if it preserved all high-dimensional structure.

17. Summary

• High-dimensional EDA organizes and understands many variables; it does not visualize every possible pair.

• Begin with an inventory, data dictionary, and quality audit.

• Group variables by domain and measurement type.

• Investigate missingness across columns and rows, including co-occurrence.

• Identify constants, near-constants, duplicates, and possible deterministic redundancy.

• Use pairwise associations with appropriate measures and sample-size context.

• Treat clustering and visual projections as exploratory aids.

• Be cautious about multiple comparisons, scaling, and high-dimensional distance intuition.

18. Assignment

Part A — Concepts

• For p = 60 variables, calculate the number of unique pairs. Explain why pairwise inspection becomes difficult.

• Explain why a correlation matrix cannot establish independence or causation.

• Distinguish duplicate columns from distinct measurements that happen to match in this sample.

• Explain why pairwise correlation sample sizes can differ with missing data.

• Describe two ways a high-dimensional dataset can be visually misleading.

Part B — Audit a wide table

Use a dataset with at least 30 columns. Document the observational unit, create a variable inventory, and report:

• Rows, columns, data types, and likely observational unit.

• Ten columns with highest missingness, including denominators.

• Rows with the largest number of missing fields.

• Constant and near-constant columns.

• Potential identifiers and duplicate columns.

• At least three meaningful domain-based variable groups.

Part C — Relationships

• Choose 8–12 numeric variables and compute Pearson and Spearman matrices.

• Compute pairwise sample sizes and identify pairs with relatively small overlap.

• Visualize a selected matrix and explain three patterns cautiously.

• Choose two categorical variables and one categorical–numeric pair for suitable analysis.

• Explain one plausible reason an observed relationship may be misleading.

Part D — Reflection

Explain why you deliberately did not inspect every possible pair. Describe your selection logic.

19. Coding project — High-Dimensional Dataset Explorer

Extend your cumulative EDA master notebook. Build on earlier data-quality, distribution, relationship, grouped-analysis, and unusual-observation modules.

• Load a wide dataset and document its observational unit, grain, source, and domain.

• Generate a reusable variable inventory with types, missingness, unique counts, and sample values.

• Create row- and column-level missingness summaries and a co-occurrence view for selected columns.

• Identify constants, near-constants, candidate identifiers, and exact duplicate columns; explain findings rather than deleting them.

• Group variables into domain categories and create a data dictionary.

• Compute Pearson and Spearman associations for selected numeric variables, including pairwise sample sizes.

• Create a readable correlation heatmap and one selected pair plot or set of small multiples.

• Include at least one mixed-type relationship analysis.

• Discuss multiple comparisons, missingness, scaling, and visualization limitations.

• Write an executive summary with patterns, data-quality concerns, and unresolved questions.

Acceptance criteria: reproducible notebook; explained variable selection; preserved original columns; visible pairwise sample sizes; readable plots; and no causal claims based solely on exploratory associations.

Suggested deliverables: updated master notebook, variable inventory, data dictionary, selected figures, and a concise Markdown report suitable for a professional EDA handoff.

20. Preview of Next Chapter

Chapter 19 — Time Series EDA

We will examine data whose order in time is essential: trend, seasonality, cycles, temporal dependence, irregular sampling, missing periods, changing variance, and structural breaks. We will distinguish event time from observation time, inspect temporal aggregation carefully, and build a practical time-series EDA workflow.

Boundary: the focus remains descriptive investigation. Forecasting models and predictive time-series algorithms are outside this EDA chapter.